

شرح پروژه:

فرض کنید یک فروشگاه اینترنتی داریم که سفارشات مشتریان را به صورت فایل‌های متنی دریافت می‌کند. هر فایل شامل اطلاعات سفارش (نام مشتری، محصول، تعداد و قیمت) است. شما به عنوان توسعه‌دهنده، باید سیستمی طراحی کنید که این فایل‌ها را از یک پوشه مشخص (ورودی) به صورت خودکار شناسایی و خوانده، زمان دریافت (timestamp) را به محتوای هر فایل اضافه کرده و فایل تبدیل‌شده را در پوشه دیگری (خروجی) ذخیره کند. همچنین سیستم باید هر ۱۰ ثانیه یک گزارش وضعیت در کنسول نمایش دهد تا نشان دهد که به درستی در حال اجراست.

علاوه بر این، سیستم باید در مواجهه با شرایط متغیر و خطاهای احتمالی، بدون توقف، به پردازش خود ادامه دهد. به این منظور، سناریوهای زیر برای پیاده‌سازی در نظر گرفته شده است:

- تلاش مجدد در صورت بروز تأخیر یا خطا در خواندن فایل: سیستم تا ۳ بار تلاش می‌کند تا فایل را بخواند و در صورت ادامه‌ی خطا، آن را به پوشه‌ی خطا منتقل کرده و پردازش فایل‌های بعدی را ادامه می‌دهد.
- ذخیره‌سازی موقت در صورت در دسترس نبودن پوشه خروجی: در صورت عدم دسترسی به پوشه‌ی خروجی، سیستم داده‌ها را به صورت موقت در پوشه‌ی جایگزین ذخیره کرده و پس از رفع مشکل، آنها را به پوشه‌ی اصلی منتقل می‌کند.
- ادامه‌ی پردازش بدون توقف در صورت مواجهه با فایل خالی یا ناقص: در صورت مواجهه با فایل‌های ورودی خالی یا فاقد اطلاعات مورد نیاز، سیستم خطا را ثبت کرده و بدون توقف، به پردازش فایل‌های بعدی ادامه می‌دهد.

برای یکپارچه‌سازی و تطبیق‌پذیری از چارچوب Apache Camel استفاده شود که هم‌زمان وظیفه‌ی اتصال و هماهنگی بین سرویس‌های مختلف (مانند سرویس فایل برای خواندن و نوشتن، سرویس زمان‌بندی برای گزارش‌دهی دوره‌ای و سرویس پردازشگر برای تبدیل داده و ...) را بر عهده دارد و با ارائه مکانیزم‌هایی مانند مدیریت خطا، تلاش مجدد و مسیریابی پویا، امکان تطبیق‌پذیری در زمان اجرا را نیز فراهم می‌کند. Apache Camel مسیرهای پردازش داده را تعریف می‌کند و Java برای پیاده‌سازی منطق پردازش و تبدیل داده‌ها به کار می‌رود.

ورودی:

ورودی‌های سیستم شامل فایل‌های متنی ساده‌ای هستند که هر کدام اطلاعات یک سفارش مشتری (نام مشتری، محصول، تعداد و قیمت) را در خود دارند و در پوشه‌ی data/in قرار می‌گیرند.

خروجی‌ها:

خروجی‌ها شامل فایل‌های پردازش‌شده‌ای هستند که همراه با زمان دریافت (timestamp) در پوشه‌ی data/out ذخیره می‌شوند و همچنین گزارش‌های وضعیت سیستم که هر ۱۰ ثانیه در کنسول نمایش داده می‌شوند.

ابزارهای قابل استفاده:

- (چارچوب یکپارچه‌سازی) Apache Camel
- (زبان برنامه‌نویسی) Java
- (مدیریت پروژه و ساخت) Apache Maven
- (ترسیم نمودار جریان داده) Microsoft Visio

فاز اول: برنامه‌ریزی و طراحی فرآیند

در فاز اول، تمرکز روی شناخت دقیق فرآیند یکپارچه‌سازی و ترسیم جریان داده است. شما باید با استفاده از نرم‌افزارهای ترسیم نمودار مانند Microsoft Visio یک نمودار جریان داده (Data Flow Diagram) یا فلوچارت کامل از فرآیند یکپارچه‌سازی ترسیم کنید. این نمودار باید تمام مراحل انجام پروژه را از ابتدا تا انتها به صورت گام‌به‌گام نشان دهد. همچنین در این نمودار، ورودی‌ها (فایل‌های سفارش در پوشه data/in)، خروجی‌ها (فایل‌های پردازش‌شده در data/out و لاگ‌های کنسول) و ارتباط بین سرویس‌های مختلف را به وضوح مشخص کنید. علاوه بر این، مستند اولیه‌ای از مؤلفه‌های سیستم استخراج کنید تا تصویری شفاف از نحوه کار سیستم به دست آید.

مراحل فاز اول:

- انتخاب سناریوی فروشگاه اینترنتی و تعیین دامنه یکپارچه‌سازی
- ترسیم نمودار جریان داده شامل تمام مراحل و سرویس‌ها
- شناسایی نقاط اتصال و وابستگی‌های بین سرویس‌ها
- تهیه یک گزارش توصیفی کوتاه از فرآیند طراحی شده (حداکثر ۵۰۰ کلمه)

خروجی فاز اول:

- فایل خروجی از نرم‌افزار Visio (یا تصویری با کیفیت از نمودار)
- گزارش توصیفی

فاز دوم: طراحی و پیاده‌سازی راه‌حل یکپارچه‌سازی

در فاز دوم، تمرکز روی پیاده‌سازی عملی سناریوی یکپارچه‌سازی و تبدیل آن به یک طرح اجرایی است. شما باید با استفاده از Apache Camel و Java کدهای مورد نیاز را نوشته و سیستم را تست کنید. در این فاز، علاوه بر پیاده‌سازی سناریوی اصلی، باید قابلیت‌های تطبیق‌پذیری زیر را نیز در سیستم پیاده‌سازی کنید:

- تلاش مجدد در صورت بروز تأخیر یا خطا در خواندن فایل: سیستم تا ۳ بار تلاش می‌کند تا فایل را بخواند و در صورت ادامه‌ی خطا، آن را به پوشه‌ی خطا منتقل کرده و پردازش فایل‌های بعدی را ادامه می‌دهد.
- ذخیره‌سازی موقت در صورت در دسترس نبودن پوشه خروجی: در صورت عدم دسترسی به پوشه‌ی خروجی، سیستم داده‌ها را به‌صورت موقت در پوشه‌ی جایگزین ذخیره کرده و پس از رفع مشکل، آنها را به پوشه‌ی اصلی منتقل می‌کند.
- ادامه‌ی پردازش بدون توقف در صورت مواجهه با فایل خالی یا ناقص: در صورت مواجهه با فایل‌های ورودی خالی یا فاقد اطلاعات مورد نیاز، سیستم خطا را ثبت کرده و بدون توقف، به پردازش فایل‌های بعدی ادامه می‌دهد.

مراحل فاز دوم:

- آماده‌سازی محیط توسعه (نصب Java و Maven در صورت نیاز)
- ایجاد پروژه‌ی Camel با استفاده از Maven
- نوشتن دو Route برای پردازش فایل‌ها و گزارش‌دهی دوره‌ای
- تست با حداقل سه فایل نمونه و بررسی خروجی‌ها
- تهیه مستندات کامل شامل:
 - کد نهایی به همراه توضیحات
 - اسکرین‌شات‌های خروجی فایل‌ها و لاگ‌های کنسول
 - مستند جمع‌بندی و تحلیل عملکرد

خروجی فاز دوم:

- کد کامل پروژه
- اسکرین‌شات‌های خروجی فایل‌ها و لاگ‌های کنسول
- مستند جمع‌بندی و تحلیل عملکرد (Word یا PDF)

فاز سوم: طراحی و تدوین فیلم آموزشی سیستم‌های نرم‌افزاری مقیاس وسیع

در این فاز، دانشجویان به صورت یک نفره موظف هستند یک فیلم آموزشی حداقل ۹ و حداکثر ۱۰ دقیقه‌ای از مفاهیم مورد تدریس در کلاس تهیه نمایند. ضبط فیلم به کمک ابزارهای ضبط کننده انجام شده و محتوای مورد ارائه در قالب یک فایل پاورپوینت صورت خواهد گرفت. موضوعات مدنظر بر اساس مفاهیم تدریس شده در کلاس بوده و فیلم تهیه شده در سکوه‌های مختلف فضای مجازی و حساب‌های کاربری مربوط به آزمایشگاه مرجع معماری سازمانی سرویس‌گرا قرار خواهد گرفت. با کسب رضایت قبلی، اطلاعات ارائه دهنده هر فیلم در توضیحات قرار گرفته تا موقعیت‌ها و فرصت‌های شغلی به انضمام شناخت دانشجو در فضای معماری سازمانی صورت گیرد.

لینکدین - اینستاگرام - تلگرام - وبسایت